Networks and Graph Visualization

Network data, layouts, and matrix views

With an implementation appendix

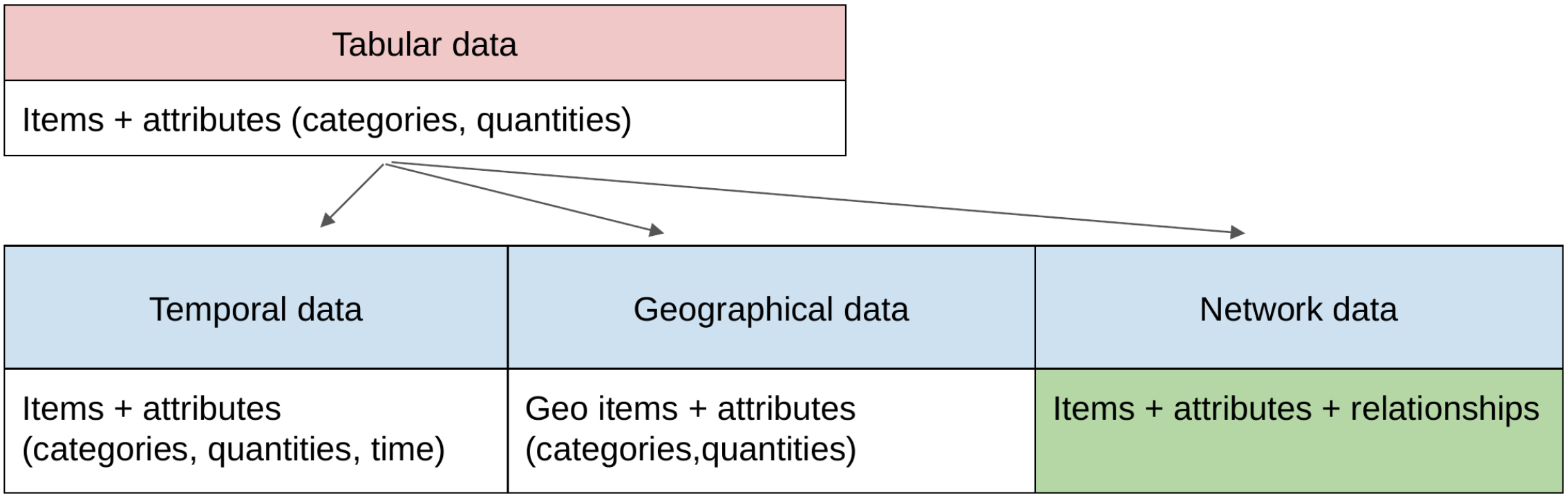
Lecture 18 · Week 10

Agenda

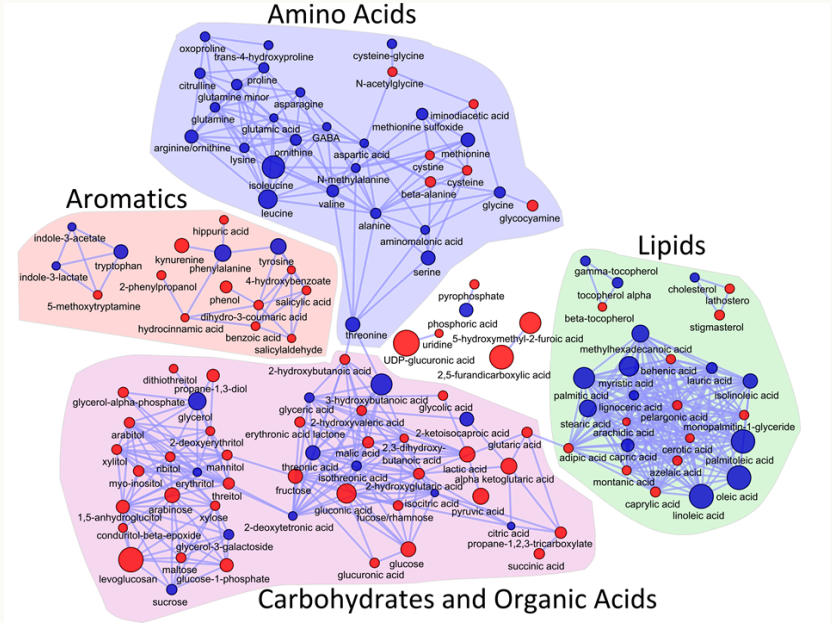
01	**Network data**	Nodes, edges, and attributes.
02	**Node-link layouts**	Fixed and force-directed placement.
03	**Matrix views**	Read, reorder, and compare.
04	**Summary**	Recap network visualization choices.
05	**Appendix**	D3 forceSimulation: networks and bubbles.

What is network data?

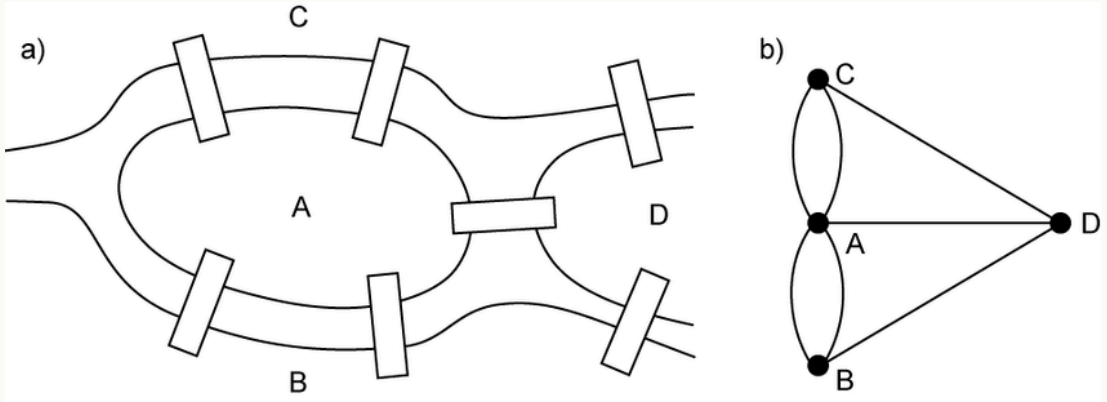
Network data describes connections or relationships between data items, together with their attributes.



Relationships are everywhere



Biochemical relationships



A spatial problem represented as a graph

- Relationships are ubiquitous across many kinds of data.
- Data attributes can define relationships through shared categories, similar values, or hierarchical membership.
- Visualizing these relationships supports analysis and problem solving.

The components of a network dataset

A network dataset can always be split into two tables: **nodes + edges**

- Items → nodes (encoded as point marks)
- Relations → edges (encoded as lines connecting two points)

Nodes

label	value
A	5
B	10
C	46
D	4
E	1

Edges

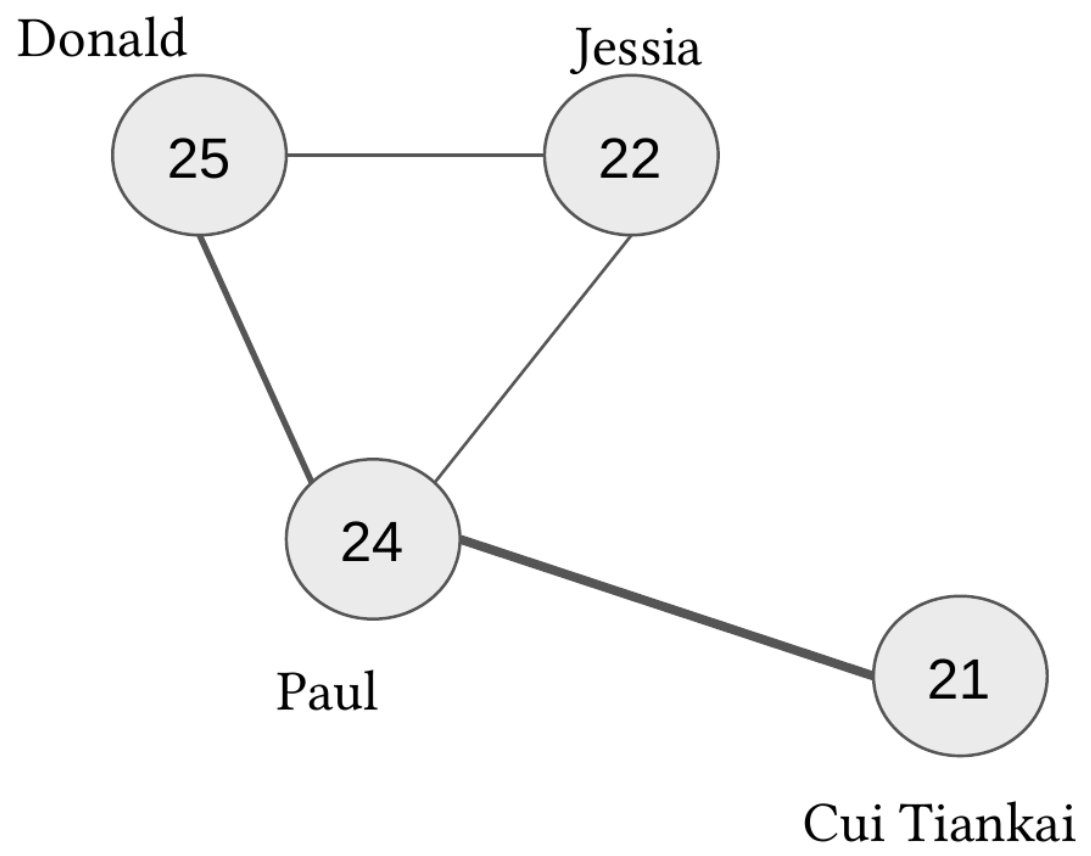
N1	N2	value
A	B	4
A	D	72
B	E	76
B	C	14
B	D	24
...	...	...

Every row in the network dataset is a link

Example: **Game of Thrones**

- Nodes table (CSV)
- Edges table (CSV)

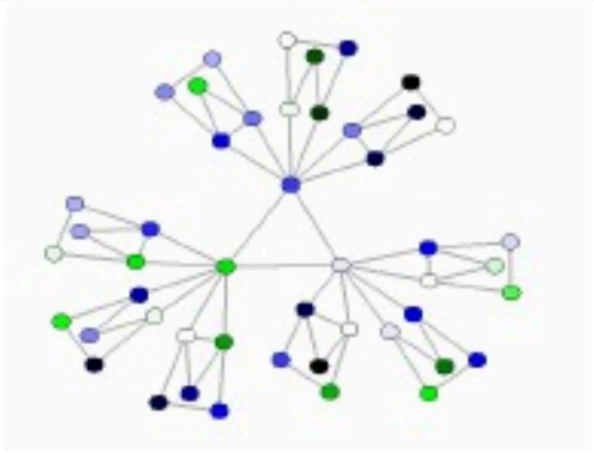
Example: a friendship network

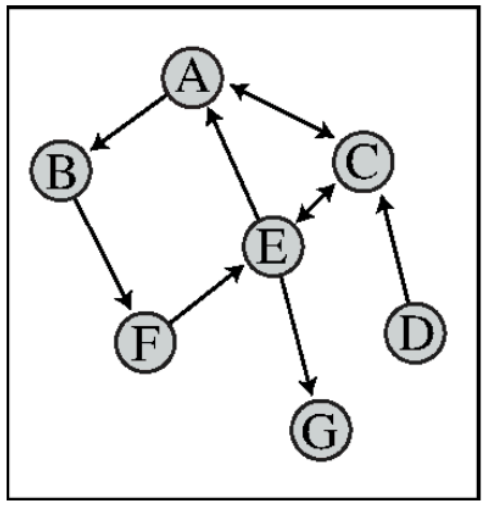


- a node: a person
- an edge exists in two persons if they are friends.
- some values can be associated to the nodes and edges
  - e.g., number in the node → age; width of the line → strength of the connection

Visualizing network data

- Node-link diagrams
  - Fixed layouts
  - Force-directed layouts
- Matrices



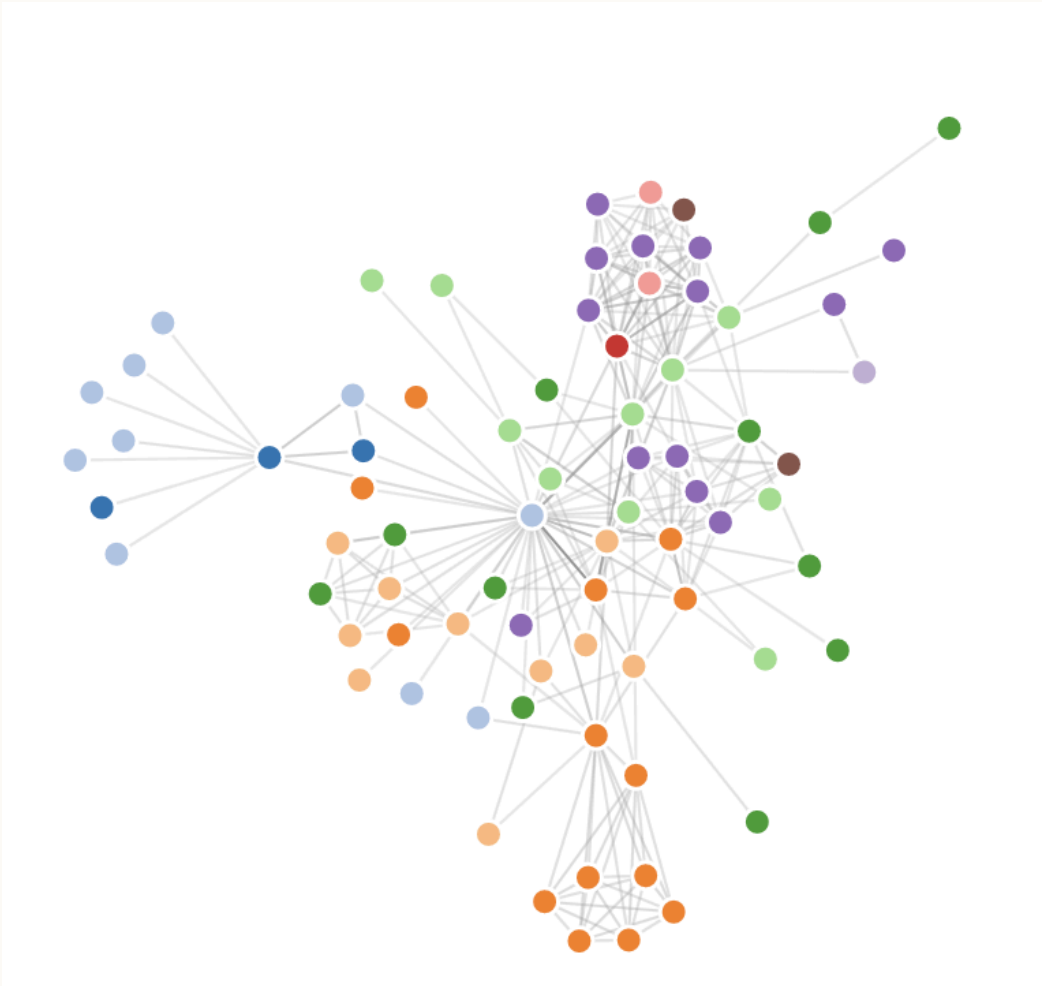


	A	B	C	D	E	F	G
A	0	1	1	0	0	0	0
B	0	0	0	0	0	1	0
C	1	0	0	0	1	0	0
D	0	0	1	0	0	0	0
E	1	0	1	0	0	0	1
F	0	0	0	0	1	0	0
G	0	0	0	0	0	0	0

Node-link diagrams

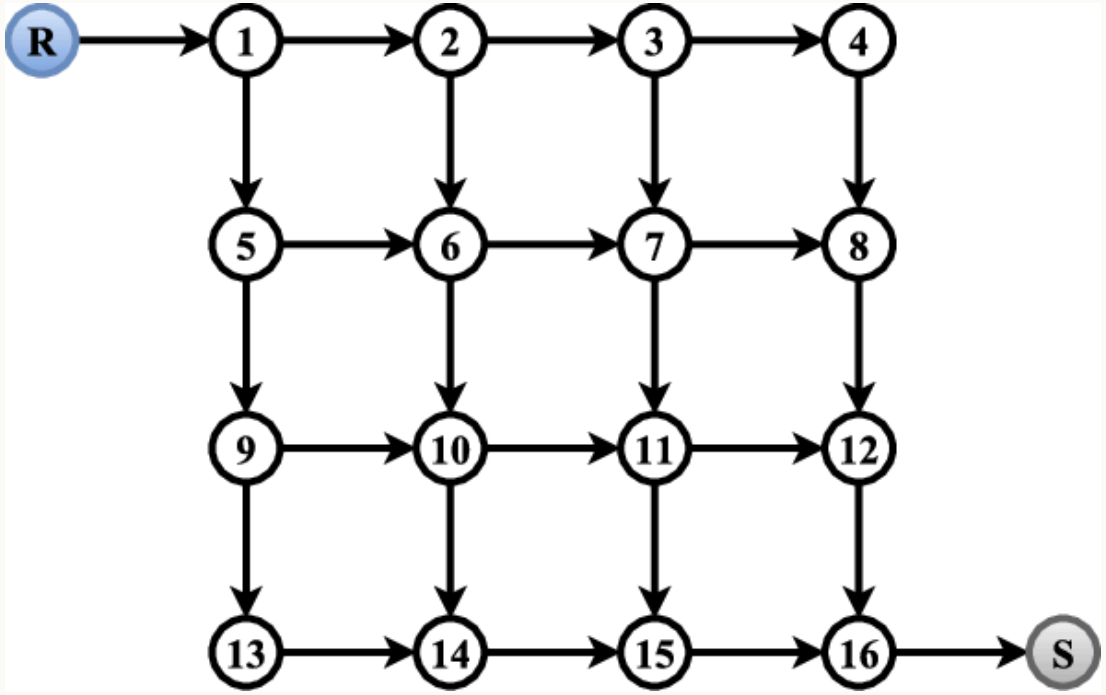
A **node-link diagram** represents nodes as point marks and edges as lines or curves connecting their endpoints.

- Follow a line to identify a relationship.
- Follow connected lines to trace a path.

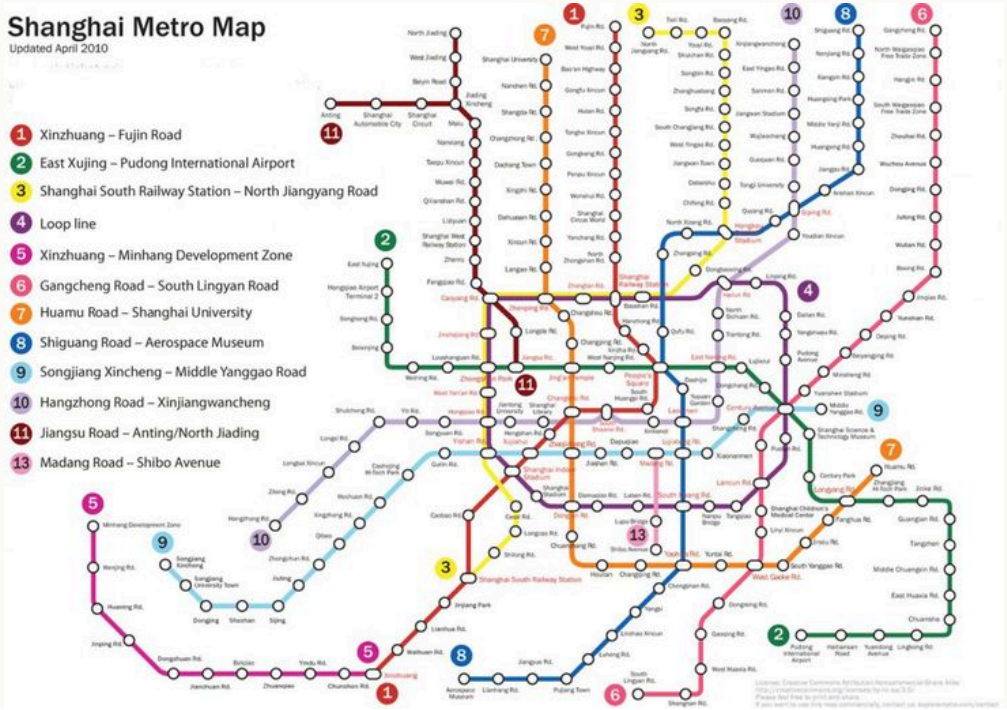


Mike Bostock / D3 · Force-directed graph

Fixed layouts



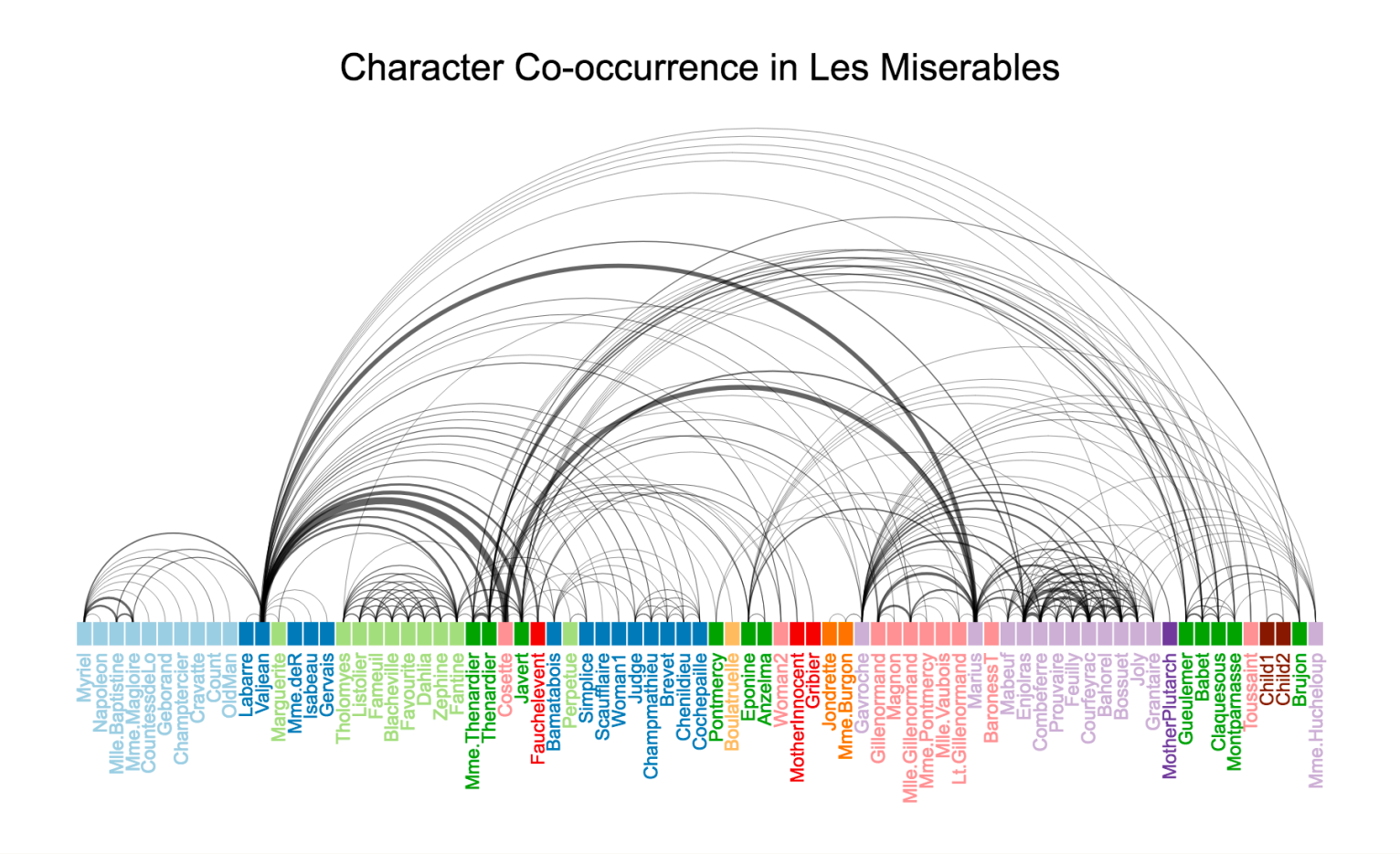
Grid arrangement



Shanghai Metro schematic

- Nodes are assigned positions: circular, linear, grid, or another useful arrangement.
- These positions need not reflect the real network structure. Designers can arrange the nodes according to the design task.

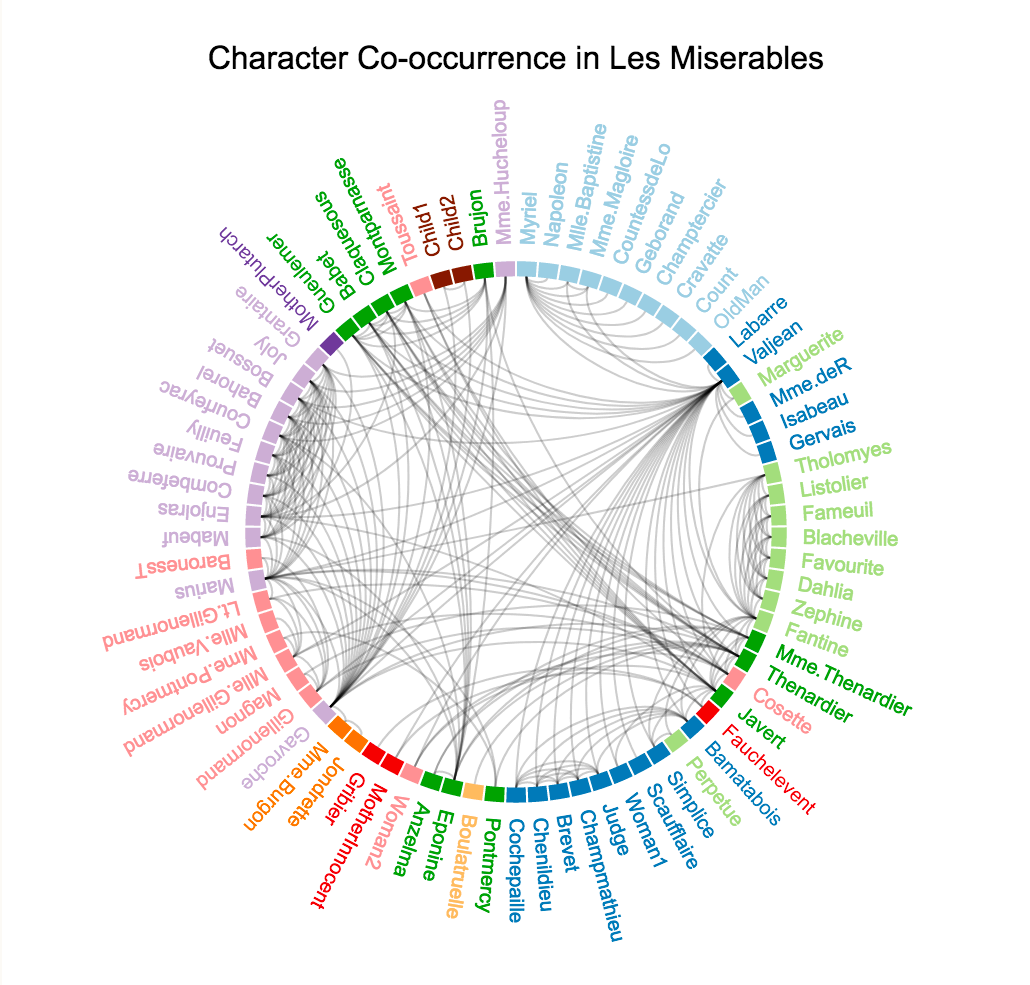
Fixed layout: character co-occurrence



Les Misérables · character co-occurrence

- Colors represent character groups.
- Arcs represent co-occurrence.
- Arc width represents co-occurrence frequency.
- Node positions follow an assigned order.

Fixed layouts allow many arrangements



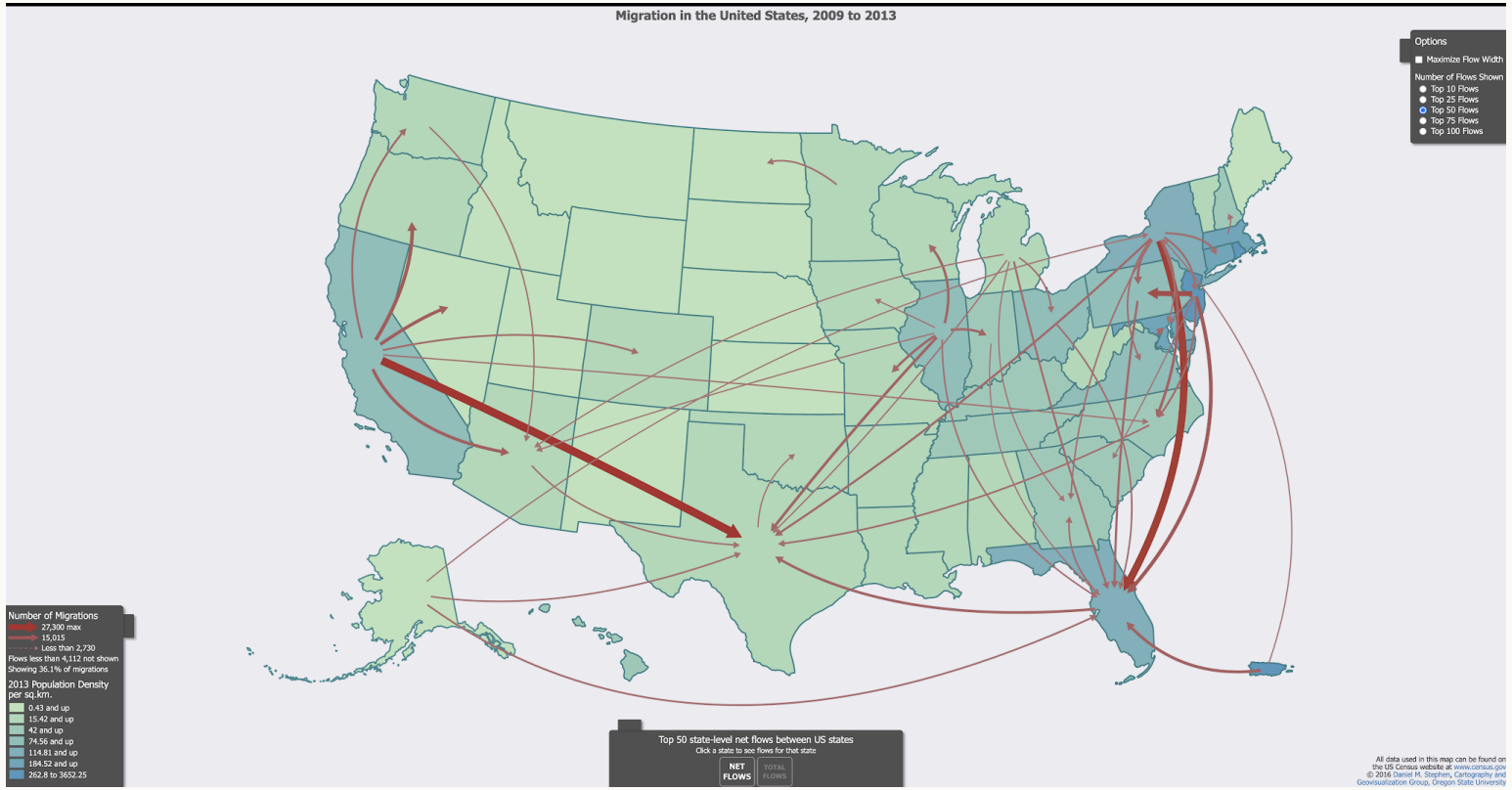
Les Misérables · circular arrangement

The same character relationships can be arranged around a circle.

Choose an arrangement that supports grouping, labeling, and comparison.

Tool example: Charticulator

Fixed layout: spatial networks



US migration map

- Nodes represent geographic regions.
- Edges represent migration flows.
- Line thickness represents the number of people.
- Position preserves geographic context.

When do we need a fixed layout?

The **structure** of the network is **not** the most important thing you want to visualize.

Advantages of fixed layout:

- Good visibility of nodes/edges because you arrange them in specific positions (feasible when the number of nodes is small).
- Nodes can be **grouped manually** into meaningful/useful categories.

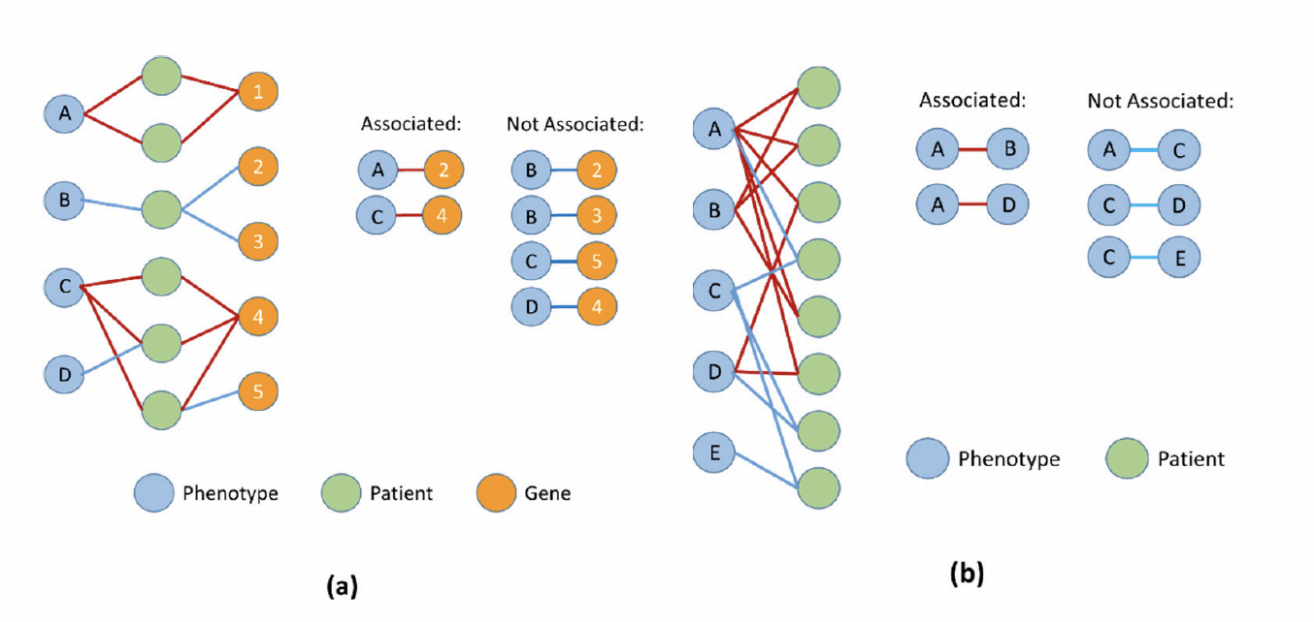
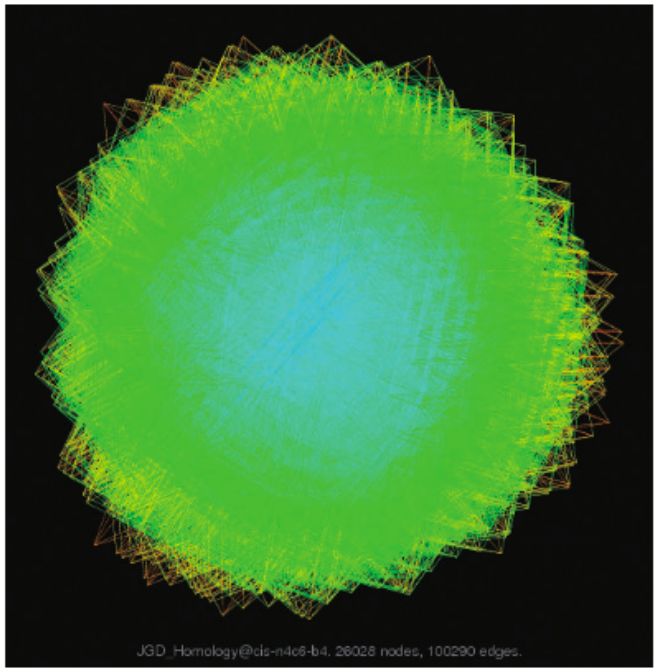
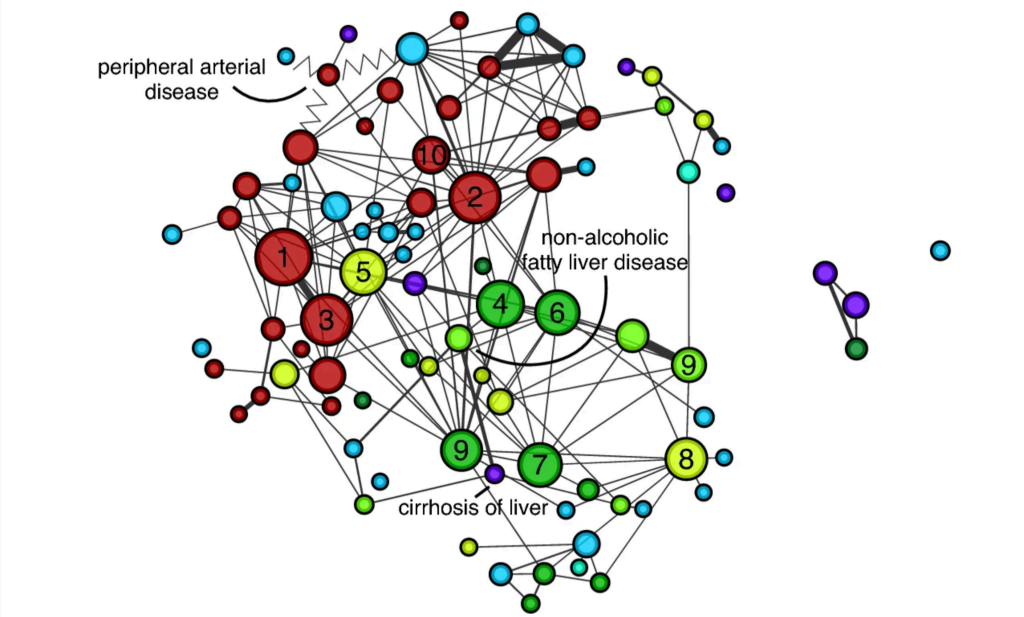


Figure 2 from *Network-Based Methods for Approaching Human Pathologies from a Phenotypic Point of View*. It visualizes relationships among phenotype–patient–gene using a graph (a fixed-layout node-link diagram).

When manual placement becomes difficult



Dense connections create a hairball



Network community 'A' characterized by vascular disorders. After removal of traits unrelated to diseases from the visualization, the most connected nodes in the community were coronary heart disease (**1**), stroke (**2**), coronary artery disease (**3**), metabolic syndrome (**4**), cardiovascular disease (**5**), hypertriglyceridemia (**6**), gout (**7**), chronic kidney disease (**8**), diabetes mellitus (**9**), and atrial fibrillation (**10**).

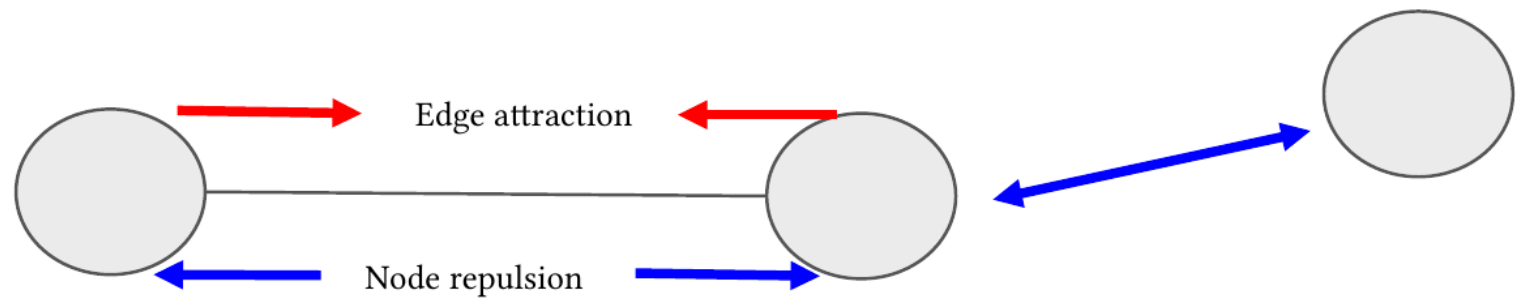
Disease relationships motivate structural exploration

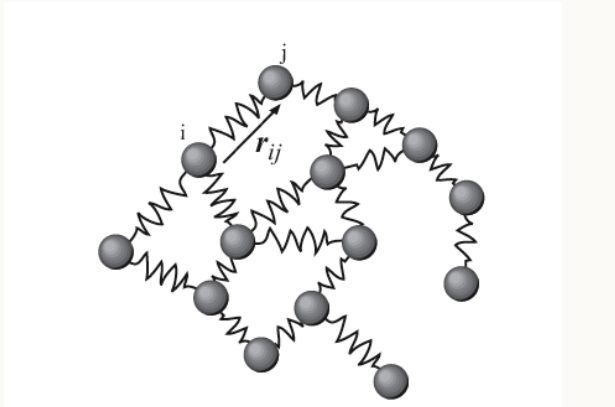
- Too many nodes and edges create clutter.
- Manually positioning nodes becomes difficult when the layout needs to reveal complex patterns of connectivity.

Force-directed layouts

A **force-directed layout** computes node positions by repeatedly applying attraction and repulsion constraints.

- **Edge attraction** brings connected endpoints toward a target distance.
- **Node repulsion** helps separate nodes, whether connected or not.
- In energy-based models, an **energy function** combines the attraction and repulsion effects.
- The algorithm repeatedly moves nodes to **reduce this energy**, seeking a balanced, readable arrangement.





A spring analogy for connected nodes

Force-directed placement: the update cycle

1. Initialize node positions.
2. Calculate the forces on each node.
3. Update node movement and position.
4. Repeat until the arrangement settles.

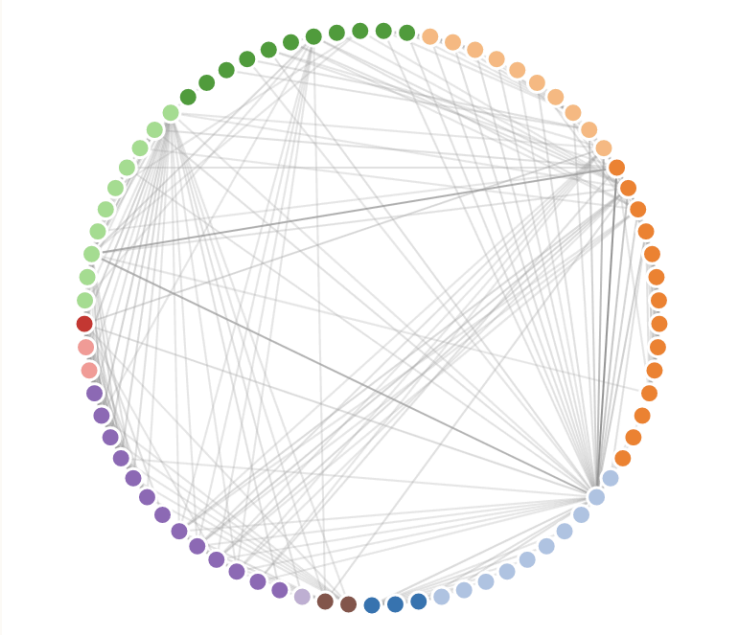
Different starting positions or settings can produce different arrangements.





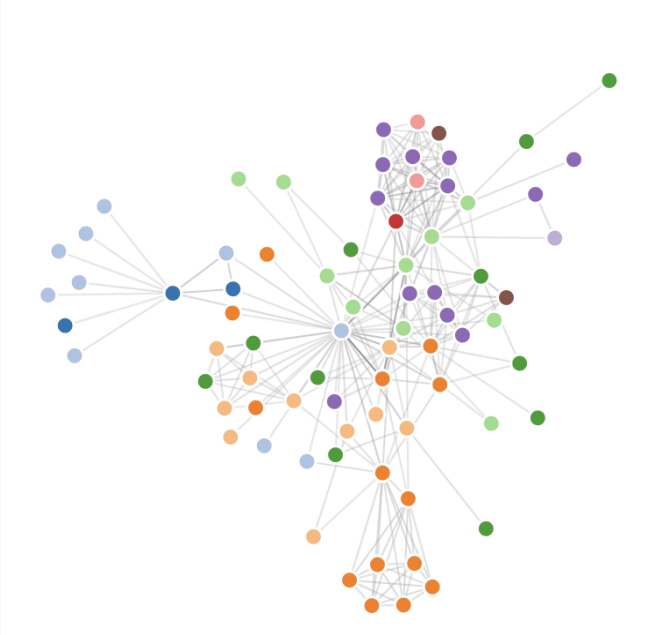
D3 force-directed graph

Force-directed layout vs. fixed layout



The circular (fixed) layout

- Node positions can be manually arranged to highlight specific relationships.
- Harder to identify the overall network structure.

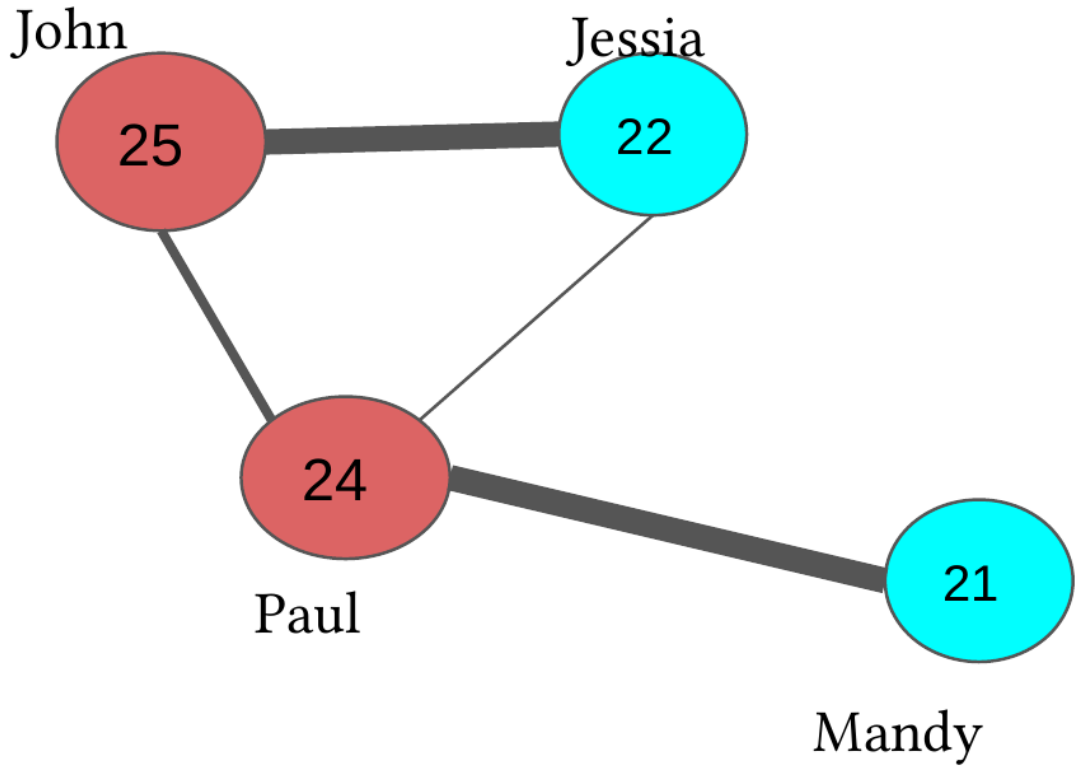


Force-directed layout

- Relationships among nodes are shown more intuitively.
- Node placement helps reveal connected neighborhoods and groups.

Les Misérables · Xianbin Gu: fixed-layout comparison · Mike Bostock / D3

Additional channels for other attributes



Nodes

- Color: categories or ordered quantities, using an appropriate scale.
- Shape: categories.
- Size: quantitative values.

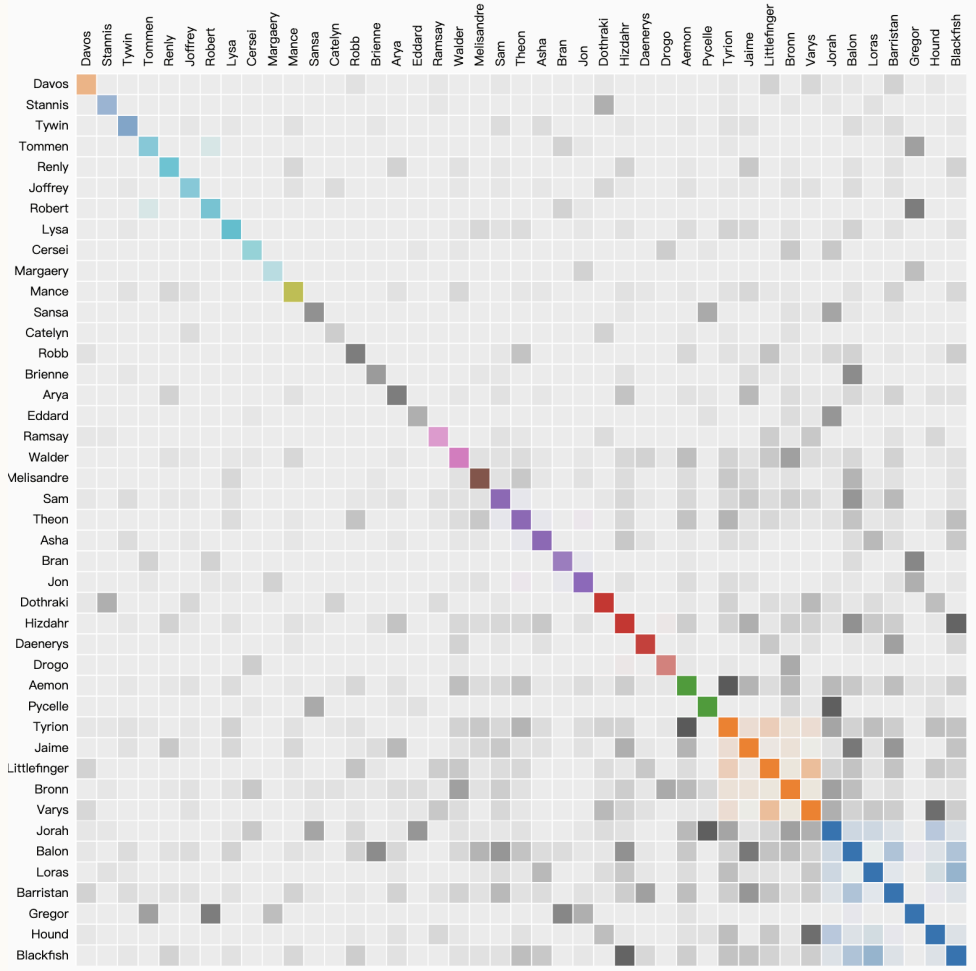
Edges

- Thickness: quantitative values.
- Line patterns: categories, such as dashed or dotted.

Matrix views

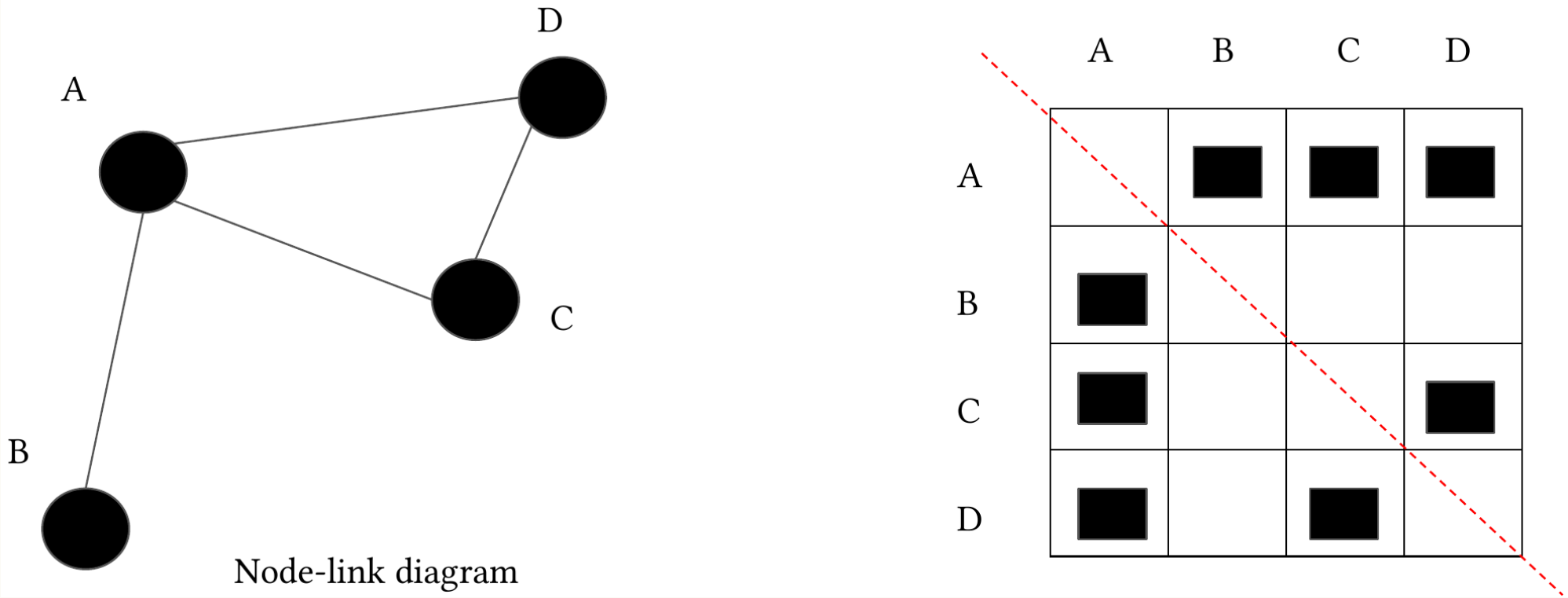
An **adjacency matrix** places the node list on both rows and columns. Each cell represents the relationship between its row node and column node.

- Nodes → rows and columns.
- Edges → marked cells.



Character co-occurrence · adjacency matrix

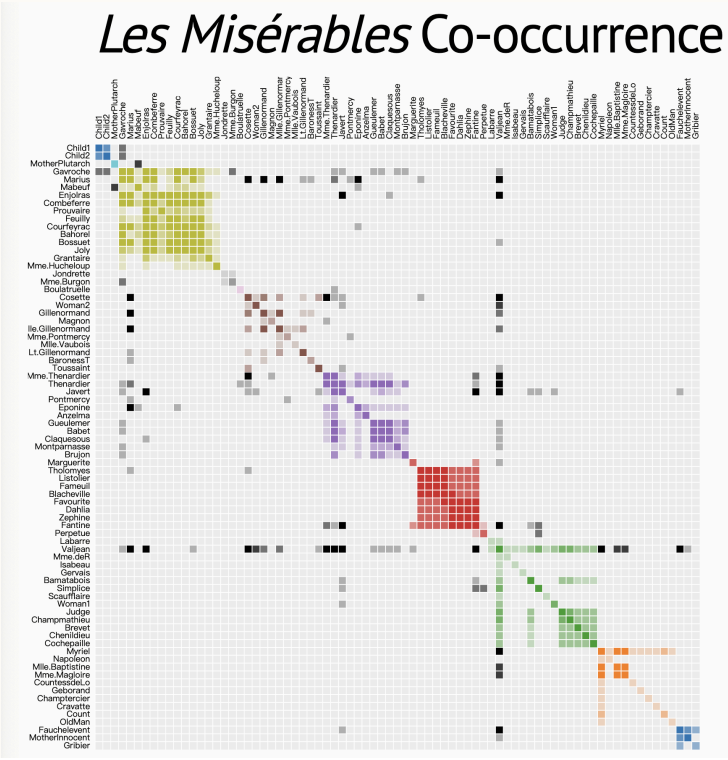
From a node-link diagram to a matrix



A mark appears only where the row node and column node share an edge.

- A connects to B, C, and D.
- C also connects to D.
- The undirected relationships appear symmetrically.

Example: Les Misérables co-occurrence



Darker cells indicate more co-occurrences. Hue marks within-group relationships; cross-group cells are black.
Les Misérables co-occurrence · interactive example

Rows and columns represent characters and use the same ordering.

- **By Name:** alphabetical order makes specific characters easier to find.
- **By Frequency:** total co-occurrence counts, highest first, highlight characters that co-occur most with others.
- **By Cluster:** placing group members together reveals dense blocks and connections between groups.

Reordering preserves network relationships, but changes which structures are easier to see.

When will we use matrix views?

Matrix views replace crossing lines with aligned cells.

Advantages

- Every node has its own row and column.
- There are no edge crossings.
- Cells support comparisons of relationship attributes.

Useful when

- Comparing many connections or edge values is central to the task.
- A node-link view has become a hairball.
- Aligned relationships matter more than following a continuous path.

Limitations of matrix views

Learning to read the view

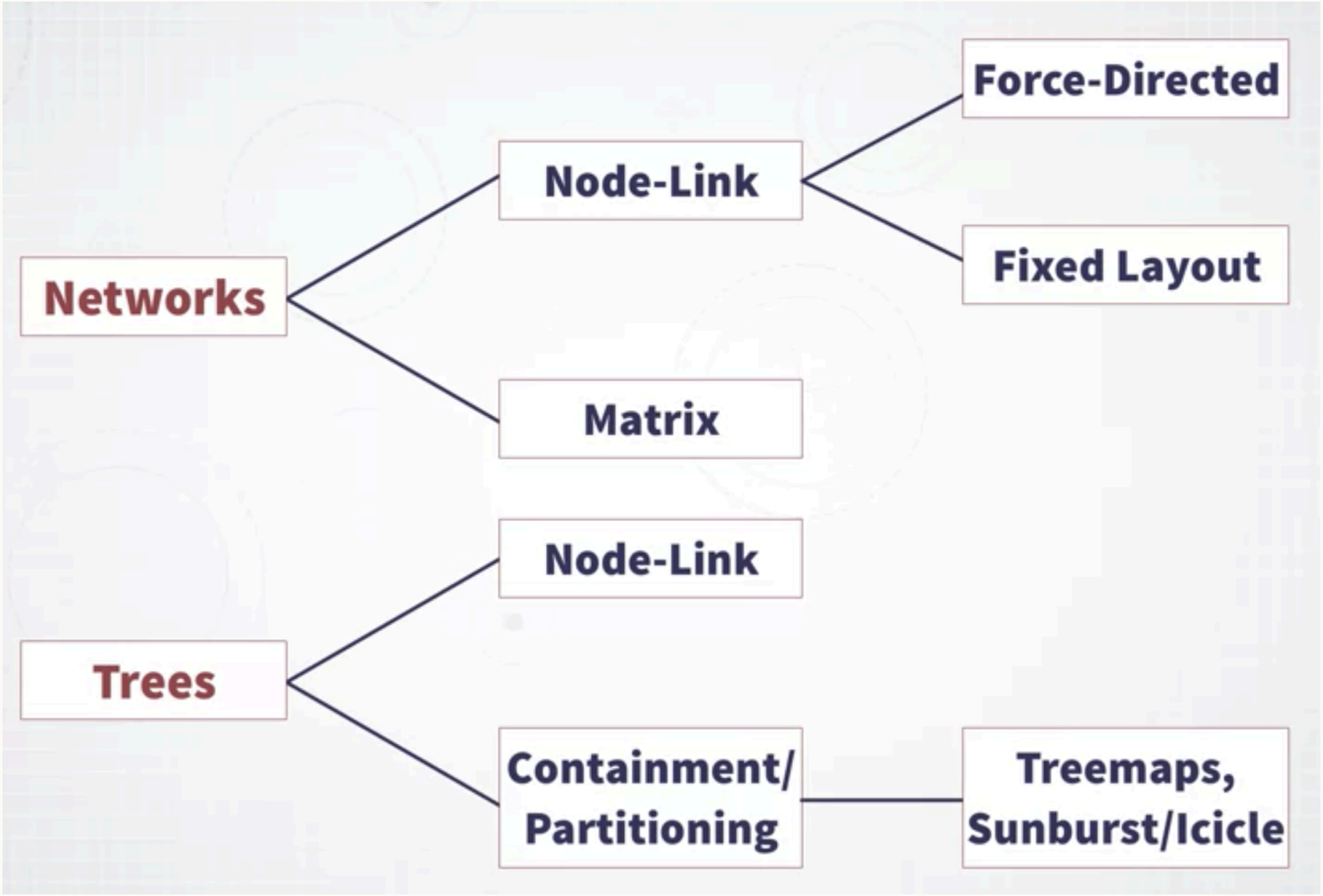
- Relationships are less immediately familiar than linked points.
- Readers must learn the row/column convention.
- Following paths requires moving between rows and columns.

Ordering matters

- An arbitrary order can hide groups and patterns.
- Reordering by known groups or clustering can expose structure.
- Interaction can help compare different orders.

Explore sorting: Les Misérables co-occurrence

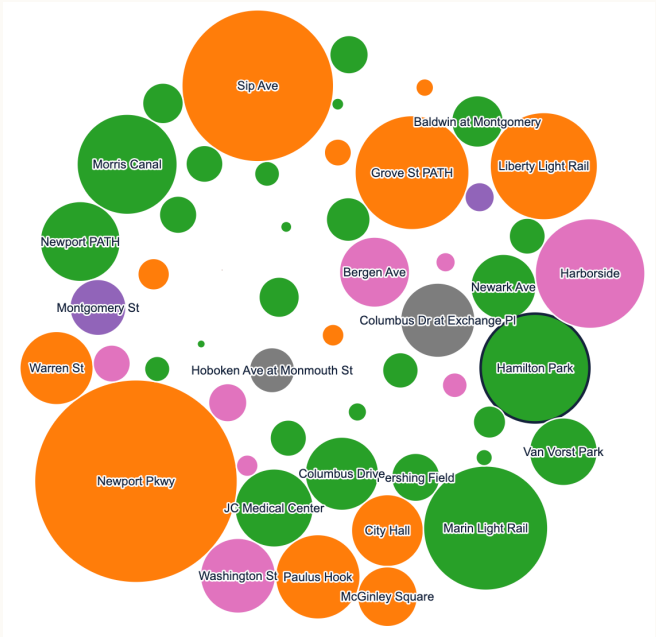
Summary

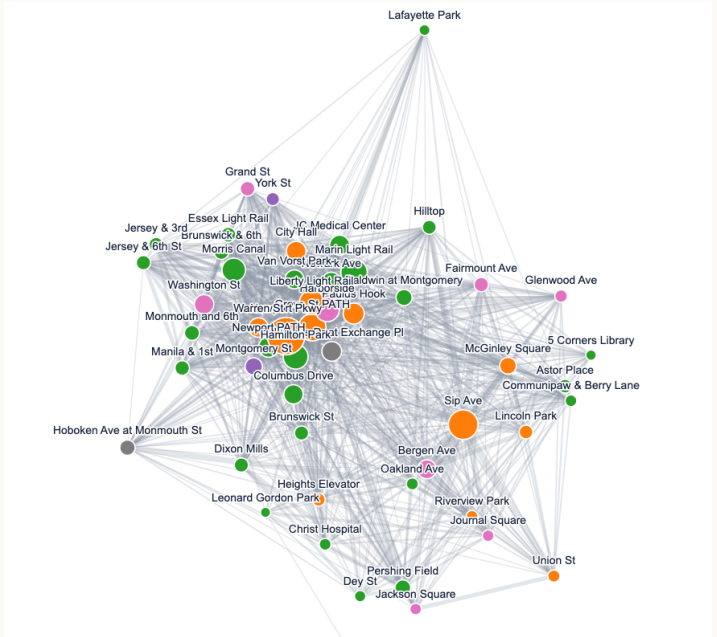

Networks
Node-Link
Force-Directed
Fixed Layout
Matrix
Trees
Node-Link
Containment/
Partitioning
Treemaps,
Sunburst/Icicle

Build a force-directed diagram

In general, building a force-directed diagram involves two operations:

1. Use `d3.forceSimulation()` to compute the positions of nodes and edge endpoints.
2. Draw nodes and edges with SVG `<circle>` and `<line>` elements.





Complete example: forceCollide.html (node-link diagram and bubble chart)

D3-force API

`d3.forceSimulation(nodes)` creates a simulation with the given array of node objects and no forces. Omitting `nodes` uses an empty array. It starts automatically: use `simulation.on()` to listen for ticks, or `stop()` and `tick(n)` to advance the layout manually.

```
const simulation = d3.forceSimulation(nodes) // Array of objects
  .velocityDecay(0.2)
  .force("x", d3.forceX(width / 2).strength(0.02))
  .force("y", d3.forceY(height / 2).strength(0.02))
  .force("collide", // Apply forces to each node
    d3.forceCollide().radius(d => radius(d.value)))
  .stop()
  .tick(1000); // 1,000 updates
```

D3-force API · forceSimulation

D3-force: Types of forces

`d3.forceLink(links)`

A link force pushes or pulls connected nodes toward a target distance. It uses the supplied array of links; omitting `links` gives an empty array.

`d3.forceCollide(radius)`

A collision force treats nodes as circles and reduces overlap using their radii. The radius can be a constant or an accessor; its default is `1`.

`d3.forceCenter(x, y)`

A centering force translates nodes so their average position reaches the specified point, preserving their relative positions. The default target is `(0, 0)`.

Link force · Collision force · Centering force

Data formats for using D3-force

Nodes

Objects with user-defined attributes; a unique key identifies each station.

```
[
  {id: 3678,
   station_name:
     "Fairmount Ave"},
  {id: 3207,
   station_name:
     "Oakland Ave"}
]
```

Links

Edge objects with `source` and `target` identifying the endpoint nodes.

```
[
  {source: "5 Corners Library",
   target: "5 Corners Library",
   frequency: 1},
  {source: "5 Corners Library",
   target: "Bergen Ave",
   frequency: 1}
]
```

Array excerpts: every endpoint must match a node in the complete nodes array. · link.id

Example: using forceLink (node-link diagram)

```
const simulation =
  d3.forceSimulation(nodes)
    .force("link", d3.forceLink(links)
      .id(d => d.id)
      .distance(d => 500 / d.value))
    .force("charge",
      d3.forceManyBody())
    .force("center",
      d3.forceCenter(width / 2,
        height / 2))
    .stop().tick(1000);
```

Inputs: nodes with `id`; links with `source`, `target`, and positive `value`.

Node identity: `.id(d => d.id)` matches endpoints to node IDs. Use `d.station_name` for name-based links.

Link distance: larger `value` means a shorter target distance.

Optional forces: charge repels nodes; center translates the network to the SVG center.

Static: `stop().tick(1000)`.
Dynamic: `.on("tick", updatePositions)`.

forceLinkedGraph-static.html · forceLinkedGraph-dynamic.html · forceLink · Manual ticks · Tick events

Example: using forceCollide

Static and dynamic bubble charts use the same forces. Examples: `forceCollide-static.html` and `forceCollide-dynamic.html`.

```
const simulation = d3.forceSimulation(nodes)
  .velocityDecay(0.2)
  .force("x", d3.forceX(width / 2).strength(0.02))
  .force("y", d3.forceY(height / 2).strength(0.02))
  .force("collide", d3.forceCollide()
    .radius(d => radius(d.value)))
  .stop().tick(1000);
```

`.radius(d => radius(d.value))` uses the visible circle radius. The collision force tries to keep two node centers at least `r(a) + r(b)` apart.

Collision radius · Static simulation

d3.drag: events and SVG updates

`d3.drag()` creates a reusable behavior that turns mouse or touch gestures into `start`, `drag`, and `end` events. Apply it to an SVG selection with `.call()`.

Attach to the circles

```
circles.call(d3.drag()
  .on("start", dragstarted)
  .on("drag", dragged)
  .on("end", dragended));
```

Each handler receives `(event, d)`: the drag event and the circle's bound node.

Redraw on every tick

```
simulation.on("tick", () => {
  circles
    .attr("cx", d => d.x)
    .attr("cy", d => d.y);
});
```

Each tick redraws node positions. For a node-link diagram, update line endpoints here too.

D3 drag API · Simulation tick events

Drag handlers in a force simulation

```
function dragstarted(event, d) {
  if (!event.active) simulation.alphaTarget(0.3).restart();
  d.fx = d.x; d.fy = d.y; // Fix the node at its current position
}
function dragged(event, d) {
  d.fx = event.x; d.fy = event.y; // Follow the drag position
}
function dragended(event, d) {
  if (!event.active) simulation.alphaTarget(0);
  d.fx = null; d.fy = null; // Release the node
}
```

`!event.active` means no other drag is active: reheat on the first start, cool on the last end.

Drag events · Fixed node positions · alphaTarget